

変数と代入

メモリモデルで理解する

rkskmt

1

関数に渡しただけで元のリストが変わる

成績データの「2倍にしたコピー」を作ろうとした。

```
Code
1 def double_all(data):
2     for i in range(len(data)):
3         data[i] = data[i] * 2
4     return data
5
6 scores = [10, 20, 30]
7 result = double_all(scores)
8 print(result) # [20, 40, 60] ← 期待どおり
9 print(scores) # [20, 40, 60] ← 元データまで変わっている
```

- **scores** には一度も代入していないのに、**元の成績が書き換わった**

🚨 今回の主題

関数に渡したリストが書き換わる仕組みを、メモリ上のアドレスで説明する。

2

リテラル（値）

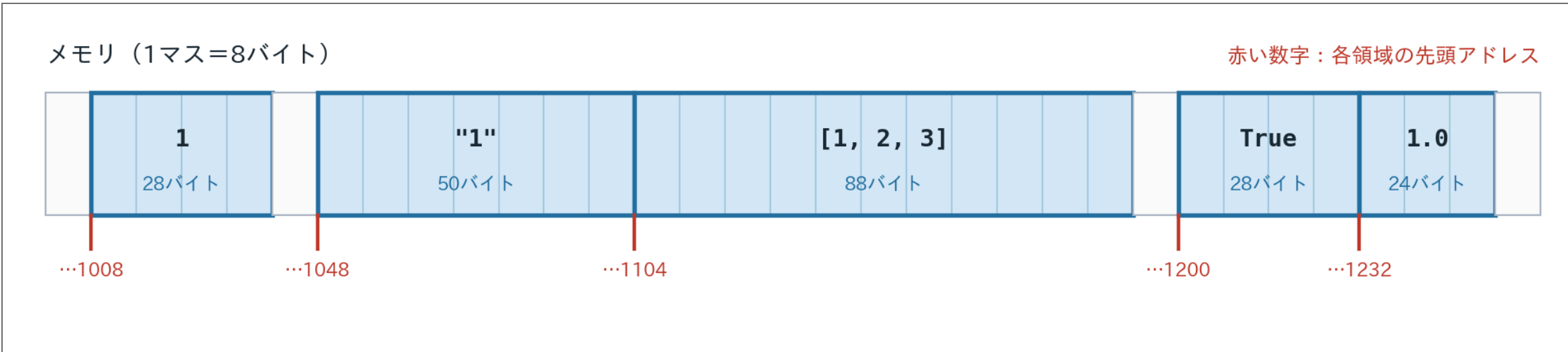
- Pythonでは**値**を直接コードに書ける。これを **リテラル（literal）** と呼ぶ
 - 値は記述された瞬間に評価可能
 - ▶ 必ずしも代入しなくていい
 - ▶ listなどでも同じ

```
Code
1 1
2 "1"
3 [1, 2, 3]
4 True
5 1.0
```

3

リテラルはメモリに確保される

- リテラルを書いた行が実行されると、
 - メモリのどこかに、そのリテラルを記述可能な領域を確保
 - その領域に値が記述される
- メモリの場所には番号が振られている。
 - これは **アドレス (address)** と呼ばれる



4

変数

一方、以下はエラーになる。なぜ？

```
Code
1 a # NameError: name 'a' is not defined
```

- **1** や **"1"** は **リテラル (直接記述できる値)**
- **a** は値ではなく **変数** として認識される

どの変数も指していない値：すぐ回収されて消える

変数が指している値：残り続ける



- 変数とは、**値のアドレスを指して、その値を使い続けるための名前**
 - 変数が指している限り、値は消えない
- **a** はまだどの値も指していないので、エラーになる

5

変数の命名規則

変数として認識される単語のルールは以下のとおり。

例外はない — ここは覚えるしかない

- 文字 (a-z, A-Z) またはアンダースコア (`_`) で **始まる**
 - つまり数字始まりはダメ (NG例: `3200scores`)
 - 終わるのはOK (例: `scores_of_3200`)
- 2文字目以降が文字、数字 (0-9)、アンダースコアであること
- 予約語 (**`if` / `for` / `while`** など) ではないこと

Code

```
1 a01      # OK。変数
2 _aa_aa   # OK。変数
3 1a       # NG。最初が数字
4 continue # NG。予約語
```

6

代入

- **変数 = 値** と書くと「**代入**」がおこなわれる
 - まず右辺の値が評価され、メモリ上のどこかにその値が記述される
 - ▶ 実際は0と1の並び (2進数) で記録される
 - 左辺の変数は、その値が記述された先頭のアドレスを保持する
 - ▶ これが代入

Code

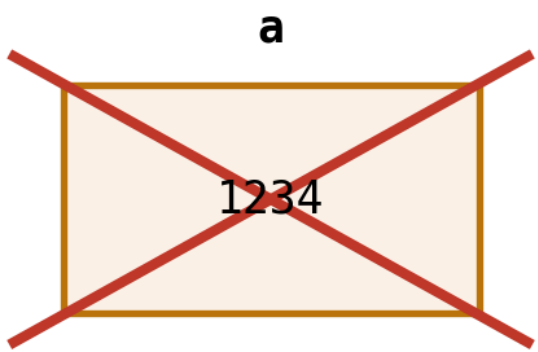
```
1 a = 1234
2 test_2 = "test2"
```

💡 変数は箱ではない

- **`a = 1234`** は「**`a`** という箱に **`1234`** が入る」 **ではない**
- **`a`** は **`1234`** が確保されたアドレスを指す
- **`a`** を使うと、指している先の値が返る

× 箱モデル: `a` という箱に値が入る

○ `a` は `1234` のアドレスを指す



`a`



`1234`

アドレス ...832

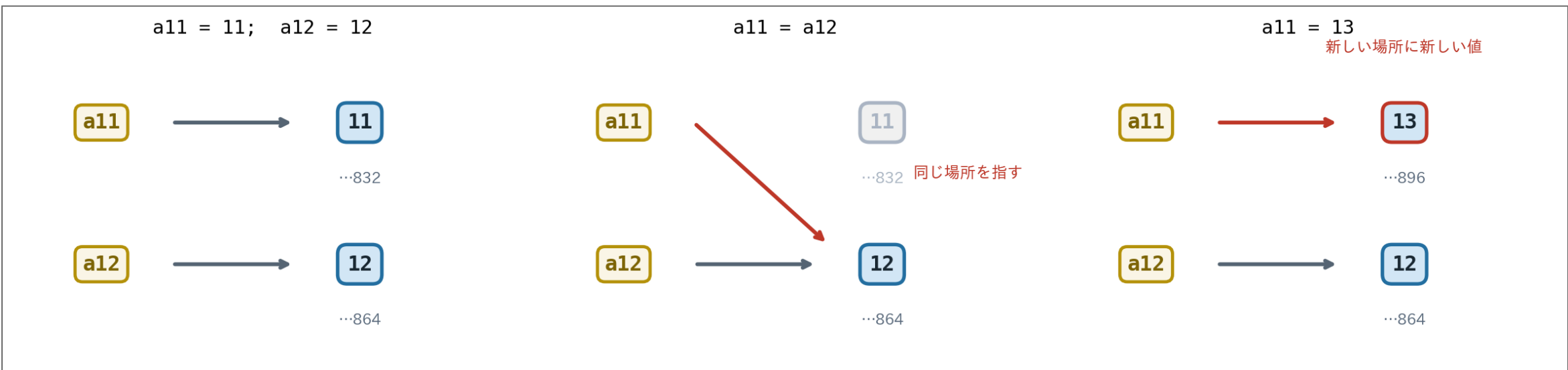
7

変数 = 変数 の代入

- **変数同士の代入**でも同じ。左辺は、右辺が指しているアドレスを指すようになる
→ つまり、2つの変数が**同じアドレス**を指す

```
Code
1 a11 = 11
2 a12 = 12
3 a11 = a12 # a11とa12は同じアドレスを指す
4 print(a12) # 12
5
6 a11 = 13 # 13 を代入
7 print(a11) # 13
8 print(a12) # ??? (同じアドレスを指しているはずだが?)
```

- 結果： **12**
- **a11 = 13** は「**a12** と共有している **12** を **13** に書き換える」**ではない**
- **13** が新しい場所に確保され、**a11** がそのアドレスを指す。**a12** が指す **12** はそのまま



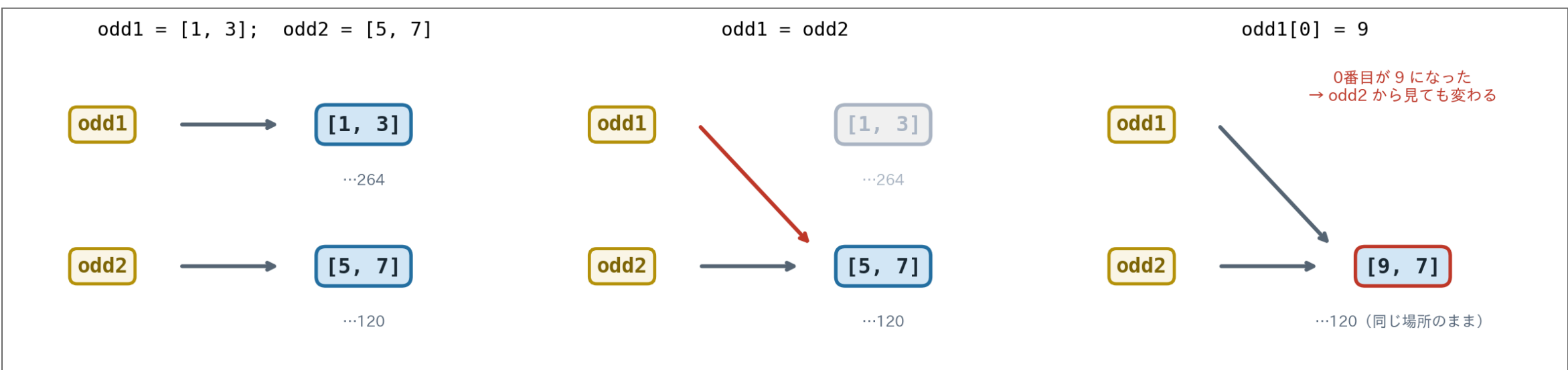
8

リストの代入（罨）

数値でやった操作をリストでやると...

```
Code
1 odd1 = [1, 3]
2 odd2 = [5, 7]
3 odd1 = odd2
4 odd1[0] = 9
5 print(odd2) # ???
```

- 結果： **[9, 7]**
- **odd1 = odd2** で、**odd1** は **odd2** と同じリストを指す
- **odd1[0] = 9** は、そのリストの0番目を **9** にする。**odd2** も同じリストを指しているの
で、**odd2** から見ても変わる



9

質問：どちらなら odd2 は無事？

ほぼ同じに見える2つのコード。 `print(odd2)` は何と出る？

```
Code
1 # A
2 odd1 = [1, 3]
3 odd2 = odd1
4 odd1 = [9, 7]
5 print(odd2) # ???
```

```
Code
1 # B
2 odd1 = [1, 3]
3 odd2 = odd1
4 odd1[0] = 9
5 print(odd2) # ???
```

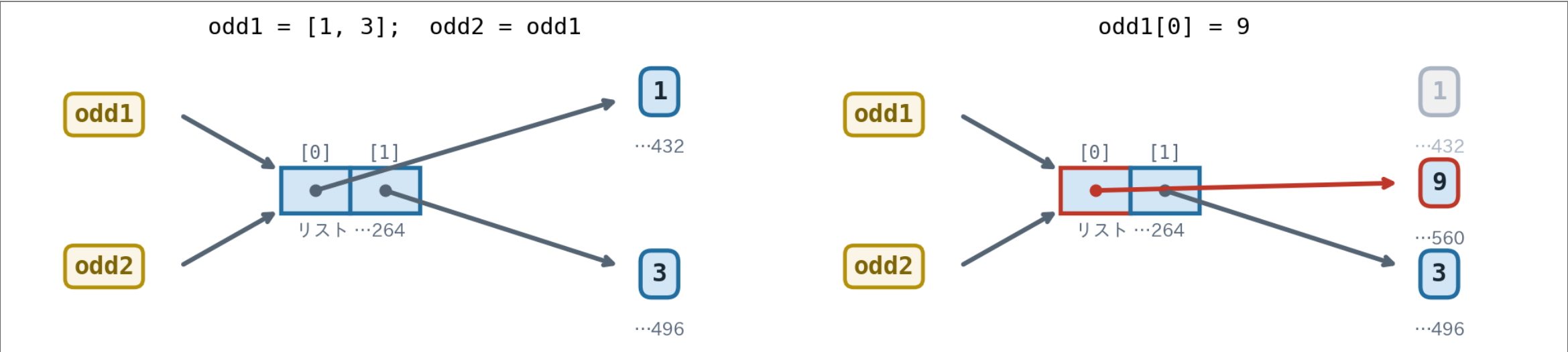
- A： `[1, 3]`。 `odd1 = [9, 7]` で新しいリストが確保され、 `odd1` はそちらを指す。
`odd2` は元のリストを指したまま
- B： `[9, 7]`。 `odd1[0] = 9` は、 `odd1` と `odd2` が指している1つのリストの0番目を変える



10

リストの要素もアドレスを指している

- `[1, 3]` を書くと、リスト本体と `1`、`3` がそれぞれ確保され、リスト本体には要素のアドレスが並ぶ
- `odd1[0] = 9` は、リストの0番目が `9` のアドレスを指すようにする代入。リスト本体のアドレスは変わらない



⚠ 要するに

左辺が変数でもリストの要素でも、代入は「左辺が右辺のアドレスを指すようにする」操作

11

id() でアドレスを確認（数値）

id() 関数で、変数が指しているアドレスを調べられる。

Code

```
1 a11 = 11
2 a12 = 12
3 print(id(a11)) # 2083050422832
4 print(id(a12)) # 2083050422864
5 a11 = a12
6 print(id(a11)) # 2083050422864。a12と同じ場所
7 a11 = 13
8 print(id(a11)) # 2083050422896。違う場所を指している
9 print(id(a12)) # 2083050422864。a12は元の場所のまま
10 print(a12)    # 12
```

- 代入のたびに、**a11** の指すアドレスが変わっている
- **a11 = 13** の後も、**a12** は同じ場所の **12** を指したまま

⚠ 豆知識：小さい整数は使い回される

- CPython は **-5~256** の **int** をあらかじめ作って使い回す
- だから **x = 11; y = 11** とすると **id(x) == id(y)** は **True** になる
- 同じ値なら同じ場所を指すことがある、というだけで、代入の仕組みは変わらない（高速化のための最適化）

12

id() でアドレスを確認（リスト）

odd1[0] = 9 の前後で、リスト本体のアドレスは変わらない。

Code

```
1 odd1 = [1, 3]
2 odd2 = [5, 7]
3 print(id(odd1)) # 2738928019264
4 print(id(odd2)) # 2738927989120
5 odd1 = odd2
6 print(id(odd1)) # 2738927989120。odd2 と同じ場所
7 odd1[0] = 9    # odd2ではなくodd1を変更
8 print(id(odd1)) # 2738927989120。変わらない
9 print(odd2)    # [9, 7]。同じ場所なので odd2 からも見える
10 odd1 = [1, 3] # 別のリテラルを代入
11 print(id(odd1)) # 2738928019328。別の場所
12 print(odd2)    # [9, 7]。odd2 はそのまま
```

- **odd1 = odd2** の後は、両者が**同じアドレスを指している**。だから **odd1[0] = 9** が **odd2** からも見える
- **odd1 = [1, 3]** では新しいリストが確保され、**odd1** はそのアドレスを指す。int のときと同じ

13

「同じ場所？」を直接聞く：is 演算子

- `id(odd1) == id(odd2)` と書く代わりに、`is` で直接確かめられる

Code

```
1 odd1 = [9, 7]
2 odd2 = odd1
3
4 print(odd1 is odd2)      # True   (同じ場所を指している)
5 print(odd1 == [9, 7])    # True   (中身が同じ)
6 print(odd1 is [9, 7])    # False  (中身は同じでも別の場所)
```

- `==` は中身の比較、`is` は場所の比較
- 2つの変数が同じアドレスを指しているかを知りたいときは `is`

ミュータブル / イミュータブル

分類	確保済みの領域の中身を変える操作	型の例
イミュータブル (immutable)	ない	<code>int</code> / <code>float</code> / <code>complex</code> / <code>bool</code> / <code>str</code> / <code>tuple</code> / <code>frozenset</code> / <code>bytes</code>
ミュータブル (mutable)	ある <code>x[0] = ...</code> <code>.append()</code> など	<code>list</code> / <code>dict</code> / <code>set</code>

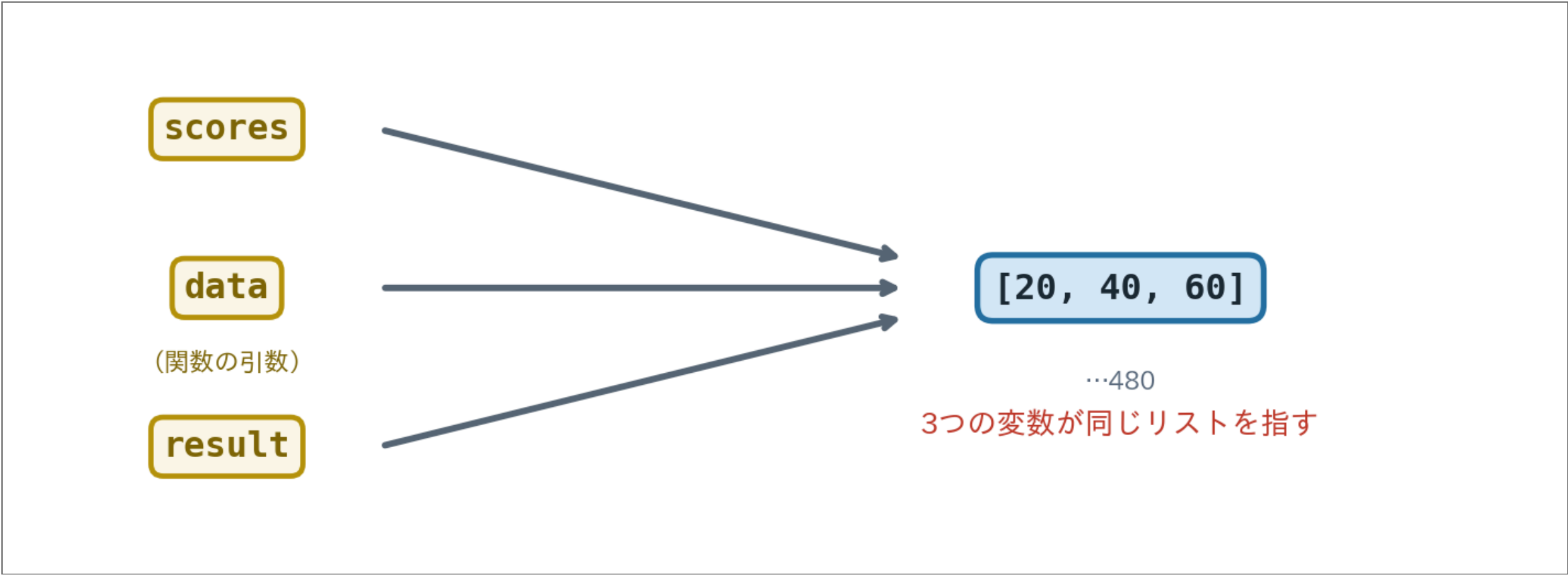
- `str` や `tuple` は `x[0]` で要素を読めるが、`x[0] = ...` はエラーになる

冒頭のコードで起きていたこと

Code

```
1 def double_all(data):
2     for i in range(len(data)):
3         data[i] = data[i] * 2 # ← リストの i 番目が新しい値を指す
4     return data
5
6 scores = [10, 20, 30]
7 result = double_all(scores) # ← data は scores と同じリストを指す
8 print(result) # [20, 40, 60]
9 print(scores) # [20, 40, 60]
```

- 関数に渡しても **コピーは起きない**。**data = scores** と代入したのと同じで、引数 **data** は **scores** と同じリストを指す
- 関数の中の **data[i] = ...** で、そのリストの各要素が2倍の値を指すようになった。だから **scores** から見ても変わっていた
- 図で表すと：



コピーで罠を回避

- リストを **別のアドレスに新たに複製** したいときは **.copy()** を使う
 - 「この場面を残したい」ときにセーブデータを別スロットへコピーするイメージ
 - 単に **=** で代入しても複製されない（同じリストを指す変数が増えるだけ）
- 冒頭の事故も、**渡す側の1行**で防げる：

Code

```
1 result = double_all(scores.copy()) # 複製を渡す → scores は無傷
```

Code

```
1 odd1 = [1, 3]
2
3 # NG: 同じアドレスを共有 → 両方が変わる
4 odd2 = odd1
5 odd2[0] = 9
6 print(odd1) # [9, 3] ← 意図せず変わってしまう
7
8 # OK: .copy() で別アドレスの複製を作る
9 odd3 = odd1.copy()
10 odd3[0] = 99
11 print(odd1) # [9, 3] ← 変わらない
12 print(odd3) # [99, 3]
13
14 # （参考）全スライス [:] でも同じ結果
15 odd4 = odd1[:]
16 print(id(odd1) == id(odd4)) # False (別アドレス)
```

odd2 = odd1 （共有）

odd3 = odd1.copy() （複製）





変数がアドレスを指すという仕組みを、次の4つの実験で確かめよ。

1. **紙に図を描く**：次のコードを1行ずつ、変数から値への矢印で描きながら追い、最後の出力を**予想してから**実行して答え合わせ

```
Code
1 a = [1, 2]
2 b = a
3 a.append(3)    # a と b はどのリストを指している？
4 a = [9]        # a と b はどのリストを指している？
5 print(b)      # ???
```

2. **dict** がミュータブルであることを **id()**（または **is**）で確認（**dict** の書き方は次回詳しく — ここでは次の2行だけ使う）

```
Code
1 d = {"名前": "田中"}    # dict を作る
2 d["点数"] = 80          # キー "点数" に 80 を代入する
```

3. **list** と同じようにアクセスできる **str** は、ミュータブルかイミュータブルかを確認
4. 上記の例で **odd1[0]** のアドレスがどうなるかを確認（**list** はミュータブルだが、**list** の要素は **int** なのでイミュータブルのはず）

▶ 解答例を見る

💡 矢印が動くのを見たいとき

Python Tutor [🔗](#) にコードを貼ると、変数から値への矢印が1行ごとに動く様子をステップ実行で確認できる。今日の図と同じ描き方。

まとめ

- リテラルを書くと、その値がメモリのどこかに確保される
- **代入** は、左辺が右辺のアドレスを指すようにする操作。コピーは起きない
- 変数同士の代入では、2つの変数が**同じアドレス**を指す
- リストの要素も値のアドレスを指している。**x[0] = ...** はその要素の指す先を変える代入
- 共有しているリストの要素を変えると、そのリストを指すどの変数から見ても変わる
- イミュータブルな値は、関数に渡しても呼び出し元から見える値が変わらない
- 事故を防ぐには **.copy()** で別の実体を渡す。確認の道具は **id()**（アドレス）と **is**（同じ場所か）